

M1.1: RAII & Guard Pattern

Theory: Binds physical resource lifetimes directly to local stack variable scopes, freeing allocations on variable destruction.

Details: Implements the 'Drop' trait. Standard 'Mutex-Guard' acts as an RAII sentinel, locking mutability on entry and unlocking the mutex automatically on exit.

Trade-off: Calling 'std::mem::forget' skips destructor runs, leaking resources permanently.

M1.2: Typestate Pattern

Theory: Represents states via generic types to validate valid interfaces at compile-time.

Details: Declares 'Connection<State>' with states 'Closed', 'Connecting', and 'Connected'. Restricts 'send()' to 'Connection<Connected>'. Transition methods consume 'self' by value.

Trade-off: Discarding and reconstructing generic objects overheads quick runtime state jumps, making classic state patterns cleaner for runtime-dominated systems.

M1.3: Type-safe Builder Pattern

Theory: Enforces mandatory field initialization at compile-time, eliminating runtime null pointer exceptions.

Details: Uses phantom types (e.g., 'Builder<HasName, HasEmail>') to track properties. The compiler blocks the terminal 'build()' invocation if any trait constraint is missing.

Trade-off: Generates complex compiler errors and nested signatures, which are best managed via derive macros.

M1.4: Newtype Pattern

Theory: Wraps base types in tuple structs to enforce semantic separation at compile-time.

Details: Encapsulates floats like 'struct Meter(f64)' and 'struct Foot(f64)' to prevent mixed calculations. Bypasses orphan rules to implement foreign traits.

Trade-off: Native operations (e.g., mathematical symbols) are blocked, requiring manual trait implementations or 'Deref' setups.

M1.5: Self-Referential Structs & Pinning

Theory: Prevents undefined behaviors by ensuring self-referential pointers remain valid when structs move in memory.

Details: Wraps pointers in 'Pin<P>' to lock memory offsets. Essential for async generator state machines where references point to local fields.

Trade-off: Bypassing Unpin markers demands unsafe pointer manipulations, which can trigger memory corruptions if mishandled.

M1.6: Interior Mutability Pattern

Theory: Allows mutating inner struct values when holding an immutable reference ('&T').

Details: Uses 'Cell' (value copies for Copy types) or 'RefCell' (tracks borrows dynamically, panicking on simultaneous read-write borrows).

Trade-off: Mismatched borrows trigger 'borrow_mut_already_borrowed_panic' crashes; concurrent code must use 'Mutex' or 'RwLock'.

M2.1: Orphan Rules & Extension Traits

Theory: Extends third-party library types with custom methods without altering upstream crates.

Details: Declares extension traits (e.g., 'pub trait MyExt') locally, implementing them on target types like 'std::string::String'.

Trade-off: Overlapping method names across extensions cause compiler errors, requiring explicit trait imports.

M2.2: Visitor AST Deserializer Pattern

Theory: Decouples data format parsing from final memory structures, enabling zero-copy deserialization.

Details: Standard in Serde. Deserializers parse formats (e.g., JSON) and invoke Visitor callbacks ('visit_str'), matching target fields.

Trade-off: Deeply nested payloads can overflow the runtime stack; parsers require strict recursion depth limits.

M2.3: Strategy Dispatch

Theory: Compares generic Strategy dispatches (monomorphized at compile-time for zero indirect pointer overhead) with dynamic strategy structures ('Box<dyn Strategy>' loaded via vtable pointers).

Details: Declares dynamic strategy structures using dyn Traits or static generic constraints.

Trade-off: Static strategy dispatches allow compiler inlinings but increase binary size; dynamic dispatches involve indirect pointer lookups.

M2.4: RAII Rollback Guard

Theory: Utilizes stack unwind mechanisms during a thread panic. Destructors check for incomplete transactions and execute automatic database ROLLBACK commands to guard data integrity.

Details: Implements TransactionGuard structs executing conditional rollbacks in drop callbacks.

Trade-off: Aborting execution paths immediately bypassing drop skips destructors entirely.

M2.5: Typestate Handshakes

Theory: Tracks network handshake processes using generic type conversions, preventing application frames from being dispatched before the connection matures.

Details: Connects transition methods over type parameters.

Trade-off: Requires extensive compile-time types mapping.

M3.2: PhantomData Markers

Theory: Advises borrow checkers on generic parameter lifetimes without incurring physical memory footprints.

Details: Declares `'PhantomData<T>'` as a Zero-Sized Type (ZST) to assert ownership bounds and lifetime variances on unused generics.

Trade-off: PhantomData is a compiler-only construct; excessive use complicates signatures and generic error tracing.

M3.6: OnceCell Singleton

Theory: Resolves compiler-static restrictions, initializing global variables safely at runtime.

Details: Uses `'OnceCell'` or `'once_cell::sync::Lazy'` to initialize variables on the first read. Enforces thread-safe write locks and locks values as read-only.

Trade-off: Singleton values are immutable post-initialization; hot-reloads require custom wrappers.

M4.3: Adapter Wrapper

Theory: Interfaces foreign libraries safely, mapping naked pointers to RAII handles.

Details: Encapsulates unsafe C pointers inside Safe C++ Adapters.

Trade-off: Generates stack nesting levels.

M2.6: Deref Coercion

Theory: Overloads Deref and DerefMut traits, directing unresolved method lookups on wrapper types to the inner wrapped object automatically.

Details: Implements Deref to Target type.

Trade-off: Overusing Deref creates anti-patterns.

M3.3: Command Invoker

Theory: Decouples command dispatchers from system targets, wrapping actions in objects to support undo/redo workflows.

Details: Declares execute and undo callbacks on commands.

Trade-off: Small structures multiply rapidly.

M4.1: Flyweight Cow Pattern

Theory: Saves heap space by keeping references in `'Cow<'a, B>'`. Automatically clones data to generate owned variables only when mutable writes are triggered.

Details: Wraps variables in Cow types.

Trade-off: High write densities make Cow lookups redundant.

M4.4: Bridge Component

Theory: Separates abstract specifications from physical execution structures.

Details: Holds pluggable dyn traits inside context models.

Trade-off: Adds indirect pointers.

M4.5: Decorator Wrapper

Theory: Wraps readers in functional layers to enrich runtime features dynamically.

Details: Implements target traits recursively.

Trade-off: Deep structures are hard to debug.

M3.4: Observer Event Bus

Theory: Thread-safe event distribution structures breaking reference cycles via Weak pointers.

Details: Manages listeners collections as `'Vec<Weak<dyn Listener>'`.

Trade-off: High-frequency acquisitions risk thread locking stalls.

M3.1: Lazy Iterators

Theory: Chain operations like `'map()'` or `'filter()'` retain lazy structures. The nested sequence compiles into a flat, highly optimized loop once terminal collections are triggered.

Details: Employs lazy iterators mapping items inline.

Trade-off: Nesting filters can complicate compiler lifetime resolutions.

M3.5: Abstract Factory

Theory: Decouples component allocations, resolving creations dynamically.

Details: Generates registry lookup tables.

Trade-off: Disables static optimizer loops.

M4.2: Memento Snapshot

Theory: Captures object properties without exposing private states.

Details: Packs states into immutable snapshot objects.

Trade-off: Capturing bulky datasets frequently strains memory.

M5.1: Chain Middleware

Theory: Models sequential request processors that execute checkpoints or chain invocations.

Details: Manages handlers array passing context.

Trade-off: Long chains increase latency.

M5.2: Dynamic State Trait

Theory: Transitions behavior dynamically by switching states in-place.

Details: Swaps Boxed state instances inside contexts.

Trade-off: Trait objects trigger heap allocations.

M5.3: Template Method Hook

Theory: Provides skeleton algorithms while leaving target steps open to overrides.

Details: Declares default trait methods.

Trade-off: Refactoring base traits breaks implementations.

M5.4: FFI Safe Wrapper

Theory: Intercepts thread panics crossing runtime boundaries to prevent aborts.

Details: Wraps unsafe boundaries in 'catch_unwind'.

Trade-off: Doesn't catch panic=abort configurations.

M5.5: Actor Channel Isolated

Theory: Protects task state values by processing mailbox channels sequentially.

Details: Maps MPSC queues as actor mailbox channels.

Trade-off: Payload cloning adds memory overheads.

Zone T1: Rust Patterns Core APIs

std::ops::Deref, std::pin::Pin, std::marker::PhantomData,
serde::de::Visitor, tokio::sync::mpsc, std::panic::catch_unwind

Zone T2: Borrows & Run Errors

borrow_mut_already_borrowed_panic,
self_referential_dangling_pointer_error, type-
state_invalid_transition_compile_error,
orphan_rule_implementation_denied,
ffi_boundary_panic_abort, once_cell_redefinition_failed